NNES Reference and Known-Truth Validation Tutorial

Neural Network Efficient Estimator

econirl package documentation

May 2026

RTD page: https://econirl.readthedocs.io/en/latest/estimators/nnes.html **Source:** https://github.com/rawatpranjal/EconIRL/tree/main

1 Purpose

NNES estimates dynamic discrete choice models with a neural approximation to the integrated value function. The motivation is the same structural target as NFXP, CCP, MPEC, and SEES: recover reward parameters, the implied policy, the value function, and counterfactual policies. The difference is that NNES uses a neural value function inside an NPL-style policy iteration step, so it can in principle scale beyond low-dimensional tabular state spaces.

This document is a known-truth validation tutorial. It does not use real data. The synthetic DGP supplies the true reward, transition law, value function, policy, and Type A/B/C counterfactual oracle objects. The package implementation runs on this DGP and must pass the hard structural recovery gates before the generated results are written.

2 Source Paper

The source paper is Nguyen (2025). The paper embeds a neural value function approximation in the Nested Pseudo-Likelihood framework of Aguirregabiria and Mira (2002). The key theoretical result is a zero-Jacobian or Neyman-orthogonality property: at the true conditional choice probabilities, first-order value-network approximation error drops out of the structural score.

Under the paper's regularity conditions, if the neural value approximation converges faster than $n^{-1/4}$, the structural parameter estimator is $\sqrt{n}$-consistent and efficient. The orthogonality result is powerful, but it is not a finite-sample guarantee. The implementation still has to put the neural value function, policy update, reward parameters, and Bellman level on a mutually consistent point of the model.

3 When to Use NNES

Use NNES when the model is structural, transitions are known or first-stage estimated, the reward is parametric, and the state space is large enough that exact tabular value iteration or full MPEC constraints are impractical. NNES is not a behavioral cloning shortcut: matching observed choices is insufficient unless the recovered reward and counterfactual policies also match the structural model.

Use the generated results below as the package-level recovery check for the finite-state known-truth benchmark. The shared harness checks more than the training loss or in-sample choice fit: it verifies parameter, reward, policy, value, Q, and counterfactual recovery.

4 Model

Table 1: Common DDC notation shared across all primers.

Symbol	Name	Definition
$s \in \mathcal{S}$	State	Discrete or continuous
$a \in \mathcal{A}$	Action	Finite: $\lvert\mathcal{A}\rvert = J$
s'	Next state	Drawn from $P(s' \mid s, a)$
$\beta \in [0, 1)$	Discount factor	
$\sigma > 0$	Scale parameter	Logit noise scale (= 1 by convention)
$\theta \in \Theta \subset \mathbb{R}^{d_\theta}$	Structural parameters	Unknown
$\phi(s, a)$	Feature vector	Known functions of (s, a), dimension K
$u(s, a; \theta)$	Flow utility	$\theta^\top \phi(s, a)$ (linear-in-parameters)
$P(s' \mid s, a)$	Transition kernel	Known or estimated from data
$Q(s, a)$	Choice-specific value	$u(s, a; \theta) + \beta \sum_{s'} P(s'\lvert s, a)\, V(s')$
$V(s)$	Integrated value	$\sigma \log \sum_a \exp(Q(s, a)/\sigma)$
$\pi(a \mid s)$	Choice probability	$\exp\big((Q(s, a) - V(s))/\sigma\big)$
τ	Trajectory	$(s_0, a_0, s_1, a_1, \ldots)$ from one agent
$\mathcal{D}$	Panel data	Collection of N observed trajectories

The observed data are state-action-next-state trajectories

$$\{(s_{it}, a_{it}, s_{i,t+1}) : i = 1, \ldots, N,\ t = 1, \ldots, T\}.$$

Flow payoff is linear in known features,

$$u_\theta(s, a) = \phi(s, a)^\top \theta,$$

with transition law $P_a(s, s')$, discount factor β, and type-I extreme-value scale σ. Given a continuation value V,

$$Q_\theta(s, a; V) = u_\theta(s, a) + \beta \sum_{s'} P_a(s, s') V(s'),$$

and the implied logit policy is

$$\pi_{\theta,V}(a \mid s) = \frac{\exp(Q_\theta(s, a; V)/\sigma)}{\sum_b \exp(Q_\theta(s, b; V)/\sigma)}.$$

5 NNES Estimator

Given conditional choice probabilities $P(a \mid s)$, the finite-state NPL evaluation step profiles the value representation as

$$W_\theta[P] = W_z[P]\theta + W_e[P],$$

where $W_z[P]$ is the expected discounted feature valuation and $W_e[P]$ is the expected discounted logit entropy term under the fixed CCP iterate. The implementation trains a ReLU MLP $V_\gamma(s)$ by supervised regression on $W_\theta[P]$ as the NNES value approximation, then optimizes the structural parameters through the profiled one-step NPL likelihood rather than freezing the continuation value at the previous θ. The reported validation value function is re-evaluated from the final recovered reward and policy on the Bellman scale.

Algorithm 1: NNES with NPL value regression

Input: Panel data, reward features, transitions, discount β, initial CCPs, anchor state
Estimate initial CCPs from empirical state-action frequencies
for *outer iteration* $k = 1, \ldots, K$ **do**
  Compute the NPL value target $W_\theta[P_{k-1}]$
  Normalize the target by the anchor state
  Train V_γ to predict the anchored target over all states
  Maximize the profiled pseudo-likelihood over θ through $W_z[P]\theta + W_e[P]$
  Update CCPs from the implied logit policy
end
Evaluate the final policy value on the Bellman scale
Output: $\hat{\theta}$, value function, policy, log likelihood, diagnostics

Comparison with SEES. SEES (Luo and Sang, 2024) approximates V with deterministic basis functions and penalizes the Bellman residual in a joint optimization problem. NNES uses a neural value approximation and NPL orthogonality instead. In principle, this avoids the Schur-complement information matrix needed by SEES. In the finite-state validation harness, the profiled NPL step gives an exact policy-evaluation specialization of the paper's neural approximation idea.

6 Known-Truth DGP

The validation cell is `canonical_low_action`, the same low-dimensional action-dependent structural benchmark used for NFXP, CCP, MPEC, and SEES.

Table 2: Canonical NNES validation DGP.

Quantity	Value
Regular states	20
Absorbing states	1
Total states	21
Actions	3
Exit action	2
Discount factor	0.95
Shock scale	1.0
Reward mode	action-dependent
Reward dimension	low
State mode	low-dimensional
Simulated individuals	2,000
Periods per individual	80
Observations	160,000

The validation gates check parameter recovery, reward recovery, baseline policy recovery, Bellman-level value and Q recovery, and Type A/B/C counterfactual regret. The NNES generator runs with enforced gates, so this section is generated only when all checks pass.

7 Generated Validation Results

This section is generated by `nnes_run.py` from the `experiments.known_truth` harness and the canonical known-truth DGP. Hard gates are enforced.

7.1 Pre-Estimation Checks

Table 3: Pre-estimation checks from the known-truth NNES run.

Check	Value	Status
Feature rank	4 / 4	pass
Feature condition number	4.512	pass
Transition row error	2.42×10^{-8}	pass
Observed states	21 / 21	pass
State-action coverage	1.000	pass
Action shares	0.345, 0.330, 0.325	pass
Minimum action share	0.325	pass
Exit/absorbing anchor	true	pass

7.2 Estimator Fit

The validation run uses 2,000 individuals, 80 periods per individual, and 160,000 observations.

Table 4: Known-truth NNES run summary.

Quantity	Value
Optimizer convergence flag	true
L-BFGS-B iterations	10
Log-likelihood	-174875.8093
Estimation time	37.77 seconds
NPL outer iterations	3
Final V-network loss	0.000399
Hard gates passed	11 / 11

The profiled NPL objective recovers the structural reward, policy, Bellman-level value function, Q function, and counterfactual policies within the hard known-truth tolerances.

Table 5: Parameter recovery.

Parameter	Truth	Estimate	SE	Error
`action_0_intercept`	0.100000	0.088790	—	-0.011210
`action_0_progress`	0.500000	0.521461	—	0.021461
`action_1_intercept`	0.000000	-0.022874	—	-0.022874
`action_1_progress`	-0.200000	-0.187153	—	0.012847

Table 6: Recovery metrics.

Metric	Value
Parameter RMSE	0.017850
Parameter relative RMSE	0.065179
Parameter cosine similarity	0.998240
Reward RMSE	0.010210
Value RMSE	0.019845
Q RMSE	0.023370
Policy KL	9.81×10^{-5}
Policy total variation	0.005646
Policy max state L1	0.020214

Table 7: Hard recovery gates.

Gate	Threshold	Value	Status
npl_outer_iterations	≥ 3.000000	3.000000	pass
final_v_loss	≤ 0.050000	0.000399	pass
parameter_cosine	≥ 0.950000	0.998240	pass
parameter_relative_rmse	≤ 0.300000	0.065179	pass
reward_rmse	≤ 0.080000	0.010210	pass
policy_tv	≤ 0.030000	0.005646	pass
value_rmse	≤ 0.200000	0.019845	pass
q_rmse	≤ 0.200000	0.023370	pass
type_a_regret	≤ 0.050000	0.000224	pass
type_b_regret	≤ 0.050000	0.000332	pass
type_c_regret	≤ 0.050000	0.000055	pass

7.3 Counterfactual Recovery

Table 8: Counterfactual recovery against known oracle objects.

Counterfactual	Policy TV	Policy KL	Value RMSE	Regret
Type A	0.005080	8.28×10^{-5}	0.000271	0.000224
Type B	0.005505	8.90×10^{-5}	0.000335	0.000332
Type C	0.002695	2.13×10^{-5}	0.000076	0.000055

8 Interpretation

The validation run confirms that the NNES implementation recovers the same structural objects checked for the other validated estimators: reward parameters, policy, Bellman-level values, Q values, and counterfactual policies. The key implementation detail is that the structural MLE step profiles policy evaluation through θ, matching the source paper's one-step NPL construction.

9 Reproduce

From the repository root:

```
PYTHONPATH=src:. python papers/econirl_package/primers/nnes/nnes_run.py
```

The script writes `nnes_results.tex`, `nnes_results.json`, and a full JSON copy under:

```
/tmp/econirl_nnes_primer_known_truth
```

References

Victor Aguirregabiria and Pedro Mira. Swapping the nested fixed point algorithm: A class of estimators for discrete Markov decision models. *Econometrica*, 70(4):1519–1543, 2002.

Yao Luo and Peijun Sang. Efficient estimation of structural models via sieves. Working Paper, University of Toronto, 2024.

Hoang Nguyen. Neural networks for efficient estimation of high-dimensional dynamic discrete choice models. Working Paper, Georgetown University, 2025.